Lecture 11: Recommender Systems 1: Youtube Search

Andrei Arsene Simion

Columbia University

November 25, 2025

Outline

- Recommendation Problems
- Metrics
- Design Youtube Video Search

Outline

- Recommendation Problems
- Metrics
- Design Youtube Video Search

Recommendation Problems

- Recommendations!
- General Problem: given a user's query, tell me the most relevant items
  - Amazon! Walmart! Given a search query, tell me the most relevant products
  - Document search. Given a set of documents and a list of sentences or chunks for each document, tell me the most relevant chunks and map those back to the source documents they came from
  - Given a search from a user, display relevant ads dispersed in within the search results page
  - Given a user lands on is home page, suggest some friends

Recommendation Problems

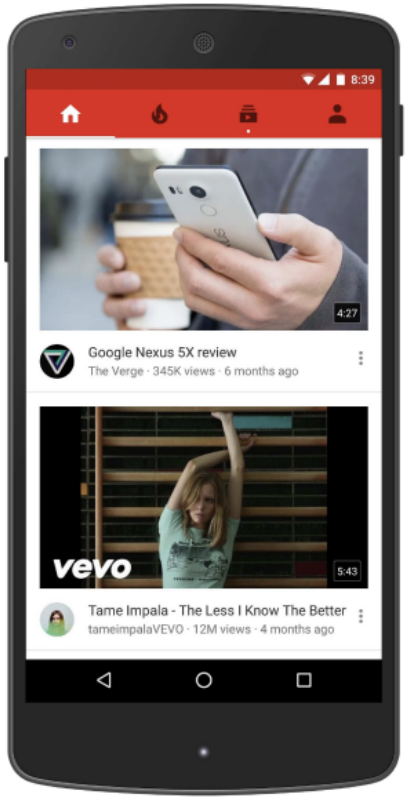


Figure 1: Recommendations displayed on YouTube mobile app home.

Recommendation Problems: What do we need to do?

- Given a problem, there are many things to consider
  - What data do we have?
  - What labels do we have?
  - What models do we use?
  - How do we know what metrics to look at? Offline metrics? Online metrics?
  - What is the scale? Is it serving fast enough?
  - Do recommendations need to be fresh upon request or can they be somewhat stale?
  - What else?

Outline

- Recommendation Problems
- Metrics
- Design Youtube Video Search

Metrics

- Let's step back a second ...
- Suppose we have a model such that, given some query, gives us the most relevant items ranked from best to worst
- How do we know that this is doing well?
- Goal: Have a metric such that if we do well we have a value of 1 if we do poorly we have a value of 0
- "Well" here means that the results returned are relevant to the user

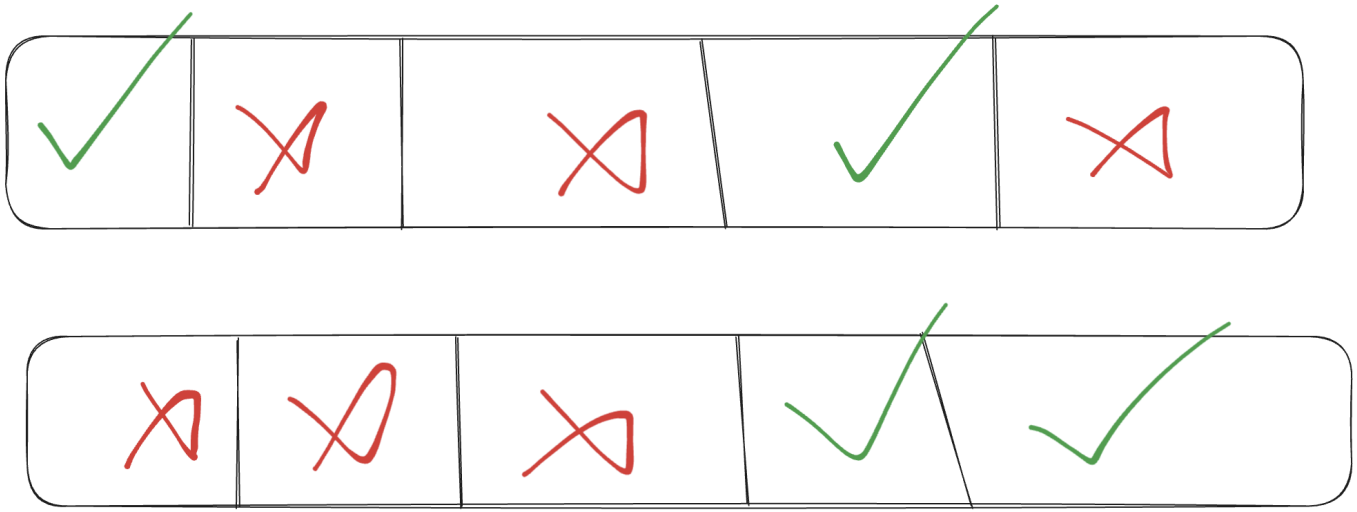
Offline Metrics: Recall

- For a certain set of queries, we might know the items that should be returned
- Suppose our system always returns 5 items ranked from best to worst
- Suppose that for a query q we have a universe of items $I(q)$ which are relevant
- For some query, this might be 1 item while for another query this might be 1000 items
- Given we made 5 ranked predictions, we can then compute

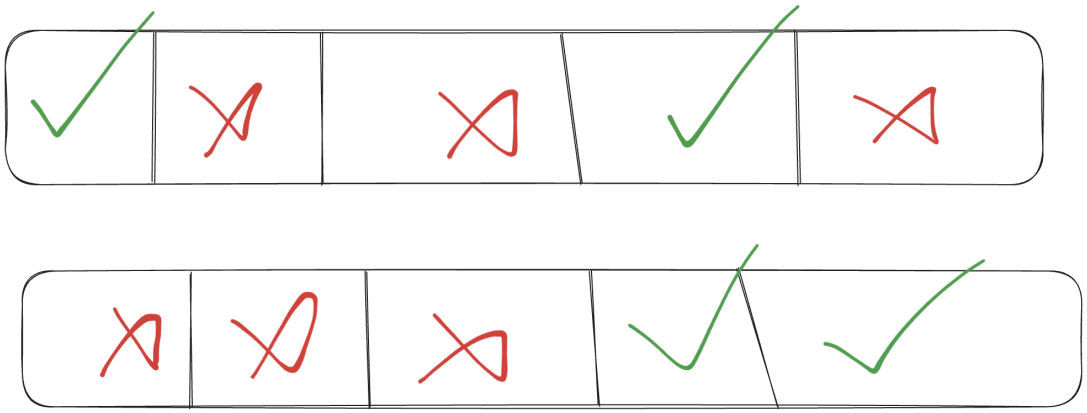
$$\text{Recall } 5 = \frac{\text{Number of relevant items among the top 5 predictions}}{|I(q)|}$$

Offline Metrics: Recall

- What's the idea: we've made 5 predictions, and we want to know of these 5 how many actually are sensible with respect to the universe of all relevant items
- But there might be a problem ... What is the Recall of the below 2 predictions?

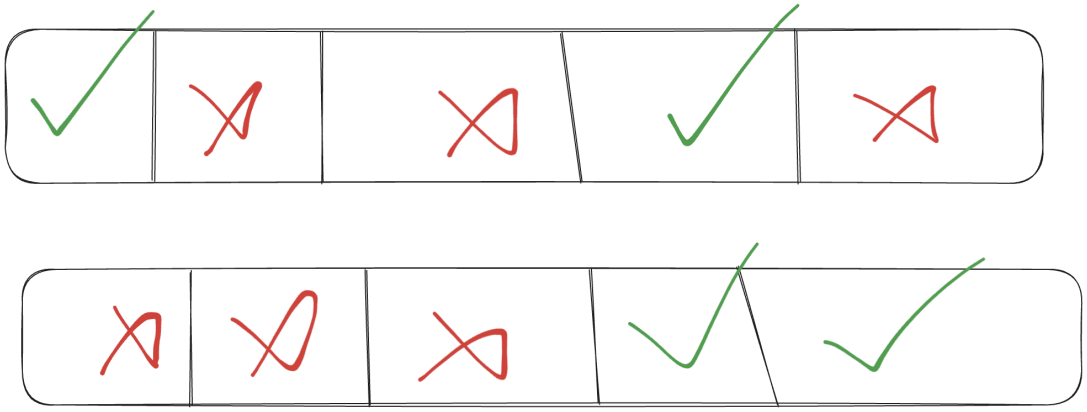


Offline Metrics: Recall



- In the above, we have 2 systems that make 5 predictions
- Each system has 2 relevant items, 3 irrelevant items
- Given the definitions, we have
  - System 1 has a Recall of $\frac{2}{|I(q)|}$
  - System 2 has a Recall of $\frac{2}{|I(q)|}$
- Both the same! But notice that
  - System 1 has a relevant item in position 1
  - System 2 has no relevant items in any of the top 3 positions
  - Moreover, what if $|I(q)| \sim 1000$ i.e. very large? In this case, the best Recall metric would be $\frac{2}{1000}$, very small!
- We should be able to differentiate better!

Offline Metrics: Recall



- So we have issues with Recall ...
  - Recall does not take into account the position of relevant and irrelevant items in the predictions - it just uses the hits count
  - If there are many irrelevant items, Recall might be hard to work with ... The maximum Recall for the above two examples is $\frac{5}{|I(q)|}$ and this maximum can be very small
- What else can we do? For the above, we can vary k and look at Recall @ k

Offline Metrics: Recall @ k

- We have the definition

$$\text{Recall @ k} = \frac{\text{Number of relevant items among the top } k \text{ predictions}}{|I(q)|}$$

- Why might we do this?
- Well, we want to know the System 1 > System 2 and we would ascertain this if we look at the Recall @1 and Recall @ 2 and Recall @ 5, for example
- System 1: $\frac{1}{|I(q)|}$, 0, $\frac{2}{|I(q)|}$
- System 2: 0, 0, $\frac{2}{|I(q)|}$
- Here, we are looking at the Recall using just the top k predictions ... Recall @ 1 being higher means System 1 puts relevant items at the front whereas System 2 only puts relevant items in positions 4 and 5

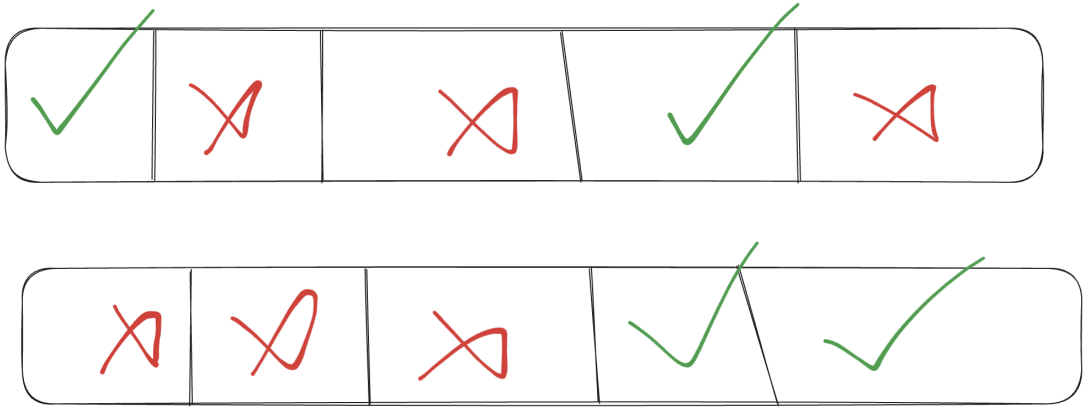
Offline Metrics: Precision @ k

- We saw with Recall that the denominator can become small ... We are making k predictions so ideally we have a metric which can be at most 1 and worst 0 ... Recall @ k can be at worst 0 and at most $\frac{k}{|I(q)|}$
- We have the definition

$$\text{Precision @ } k = \frac{\text{Number of relevant items among the top k predictions}}{k}$$

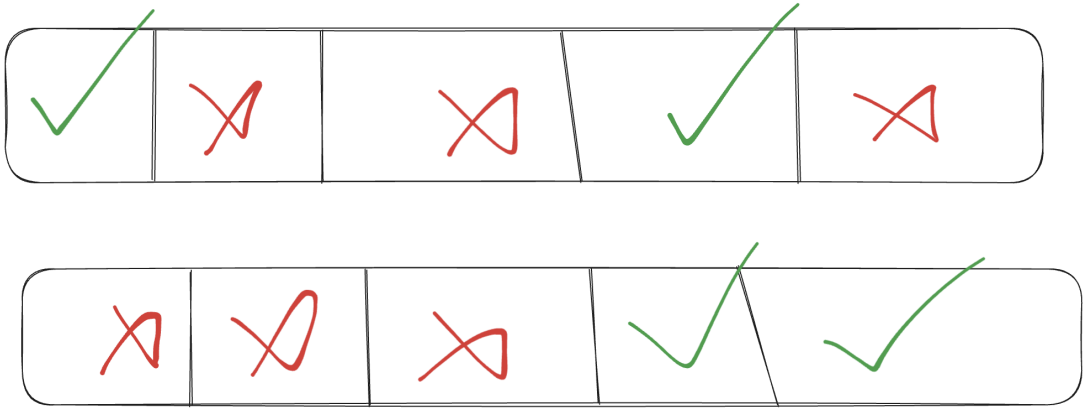
- Here, we make k predictions at at most k of them are relevant ... So the fraction above is at most 1 and at worst 0 but we are not sure if we always can touch 1

Offline Metrics: Precision



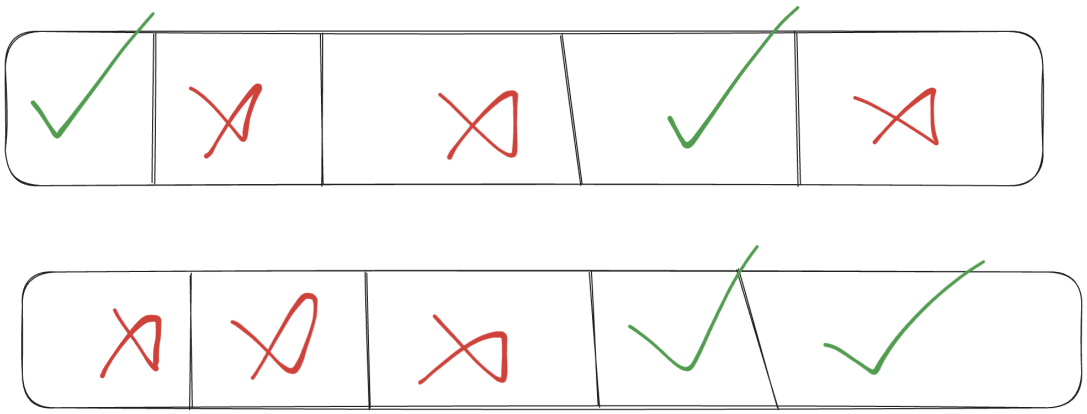
- For the two systems above, notice that Precision @ 5 is 2/5 in both cases ... What else? How do we get a better understanding?
- Like before, we need to vary @
- System 1 Precision @ 1: 1 ... The fact that this is 1 makes it easy to understand
- System 2 Precision @ 1: 0 ... The fact that this is 0 shows us how bad this system is vs System 1

Offline Metrics: Precision Issues



- Any issues?
  - As before, we have Precision @ 5 is the same for both systems ... We don't take specific position relevance into account
  - What if the number of items that can be relevant is very small, for example 2? Then, at best, the Precision @ 5 is at most $\frac{2}{5}$ ... So we have sort of the opposite problem we had with Recall

Offline Metrics: AP

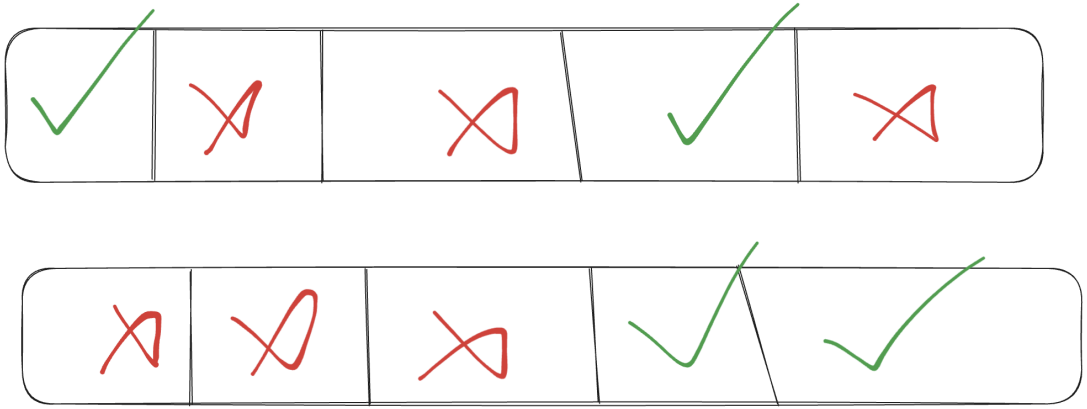


- We can get a sense of the positional performance of a system by looking at Precision @ k where k is 1, 2, ... What else?
- Absolute Precision (AP) - Mixing Recall and Precision ... Suppose rel(i) = 1/0 if the ith prediction is relevant or not ... Then we have

$$\text{AP @ } k = \frac{\sum_{i=1}^{k} \text{Precision @ } i * \text{rel(i)}}{|I(q)|}$$

- Notice that this metric is again between 0 and 1 so again we have a metric which is lowest at 0 and has the highest value 1

Offline Metrics: AP Example and Issues



- System 1: $(1 + 2/4)\ /\ 2 = 3/4$
- System 2: $(1/4 + 2/5)\ /\ 2 = 13\ /\ 40$
- With this metric, we reward the system if it puts relevant items at the front, not just if there is a hit
- Notice we could have also divided by 5 if $|I(q)|$ was this value, but then the maximum value of this metric if there are at most 2 relevant items would be $(1 + 1)\ /\ 5 = 2/10$ ... So again, this is hard to interpret

Offline Metrics: AP Issues with alternative definition

- To do Average Precision right you really should have k be dependent on $I(q)$ since the range of AP depends on $I(q)$
- For example, one query might have 5 relevant items and another might have 2
  - ... So, if you want to see how well you do on all queries you might want to consider k to be the maximum of $I(q)$ for all queries
  - Specifically, with this k, for either of the above queries, the maximum AP is 1 and you really only get rewarded if you put things at the front for all queries

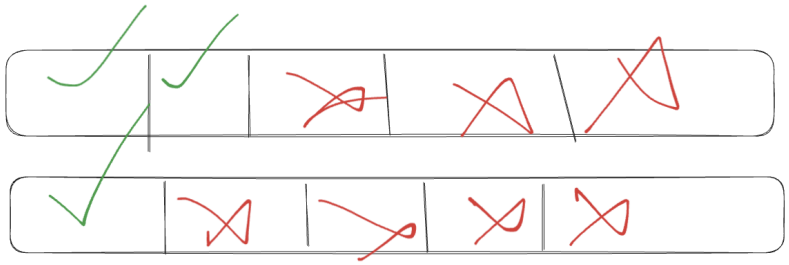
Offline Metrics: AP without $I(q)$

- To make this clear, we have that for two queries q_1 and q_2 let's says $I(q_1) = 100$ and $I(q_2) = 4$
- Let's say we have $k = 5$
- Then, maximally, the AP for these 2 queries would be $5/100$ and $2/4$, respectively
- Another definition: We can ignore $I(q)$ and divided by the total number of relevant items in the top k predictions so we don't depend on $I(q)$ ... Specifically, we can also define

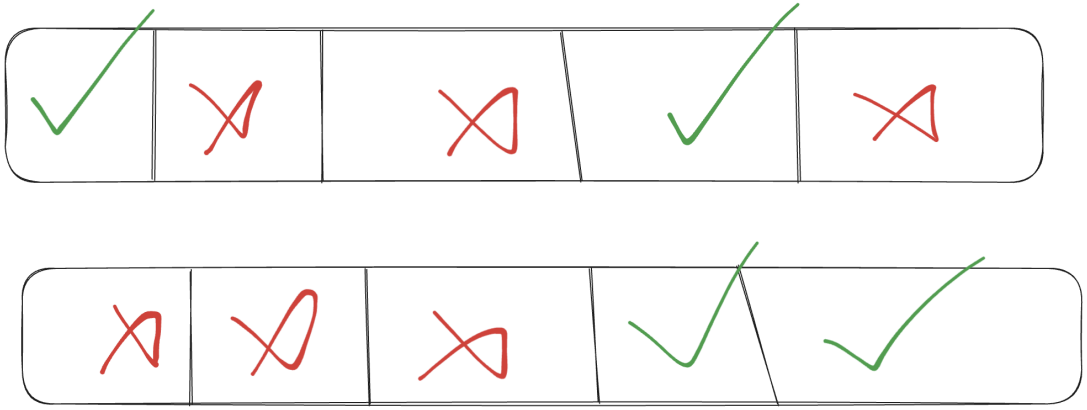
$$\text{AP @ } k = \frac{\sum_{i=1}^{k} \text{Precision @ } i \text{ * rel(i)}}{\sum_{i=1}^{k} \text{rel(i)}}$$

Offline Metrics: AP without $I(q)$

- I.e. Suppose for and we have 2 systems which produce the results below ... Which is better?
  - System 1: AP = (1+1)/2 = 1
  - System 2: AP = (1)/1 = 1
  - In this example, both systems don't put relevant items (of the 5) in the front and System 1 puts more! They are not the same!

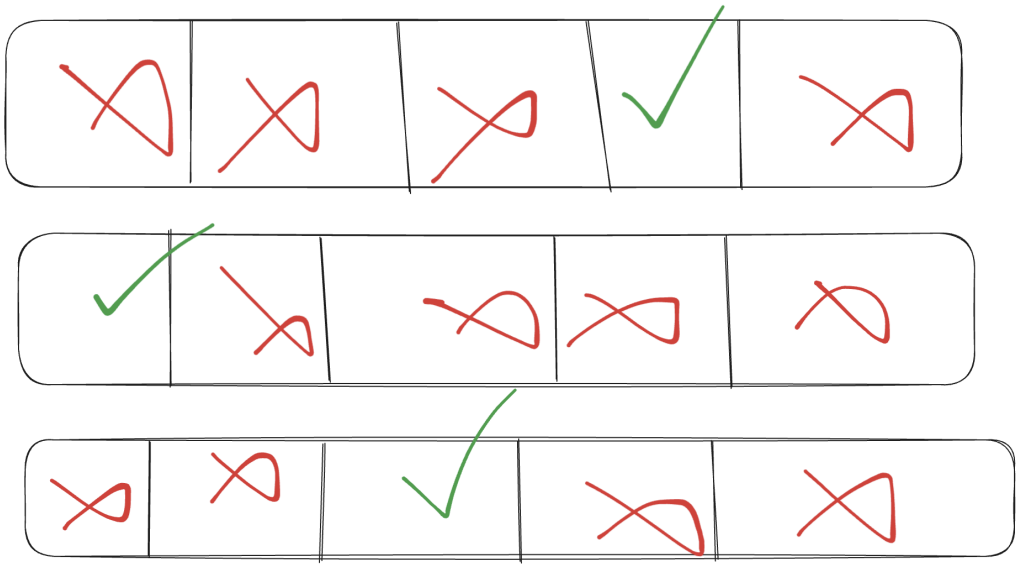


Offline Metrics: Mean Average Precision (mAP)



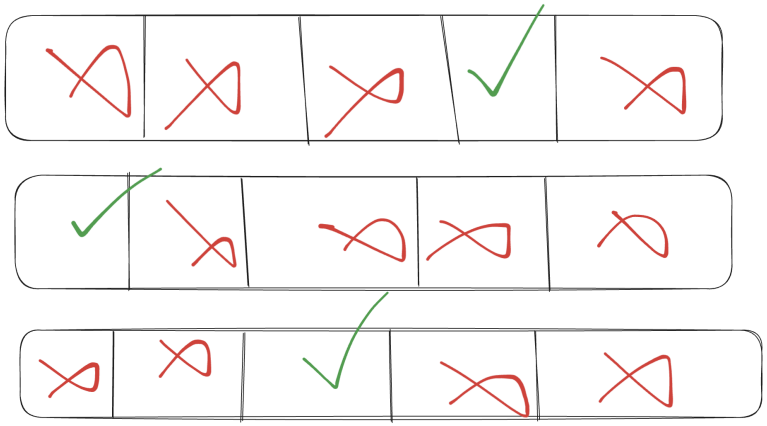
- So what if we now have a fixed System and for the 2 different queries above it returns the two ranking orders above? Can we somehow aggregate the full health of the system?
- Yes: get the AP for both queries and then take the mean
- For the above example where we predict 5 items maximally, this is (3/4 + 13/40) / 2 since these are the AP of both ranking orders and we have two queries
- Here, we assume $I(q) = 2$ is the same for both queries ...

Offline Metrics: MRR - mAP with just 1 relevant possibility



- What if we have a set results for a given set of queries and for each query we can have at most 1 relevant item?
- In this case, we can compute the Mean Reciprocal Rank, which measures how well are at putting the relevant item at the front
- Imagine search, where perhaps there is just one item we can have at the front!

Offline Metrics: MRR

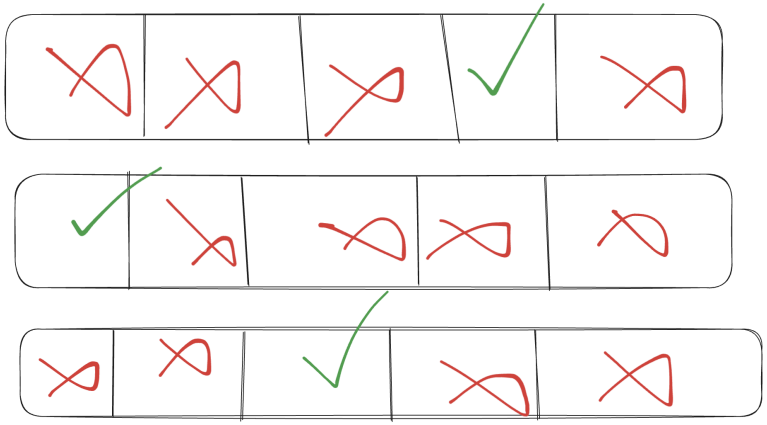


- Another example is above ... Using the second AP definition, this has mAP of (1/4 + 1 + 1/3) / 3
- Given m queries q we can name this special case Mean Reciprocal Rank (MRR @ k) as

$$\text{MRR @ } k = \frac{\sum_{i=1}^{m} 1/\text{rank}_\text{i}}{\text{m}}$$

- $rank_i$ is the rank of the first relevant result for query i in the k presented results

Offline Metrics: MRR - Don't use it if you can have more than 1 relevant item



- Given m queries q we can name this special case Mean Reciprocal Rank (MRR @ k) as

$$\text{MRR @ } k = \frac{\sum_{i=1}^{m} 1/\text{rank}_\text{i}}{\text{m}}$$

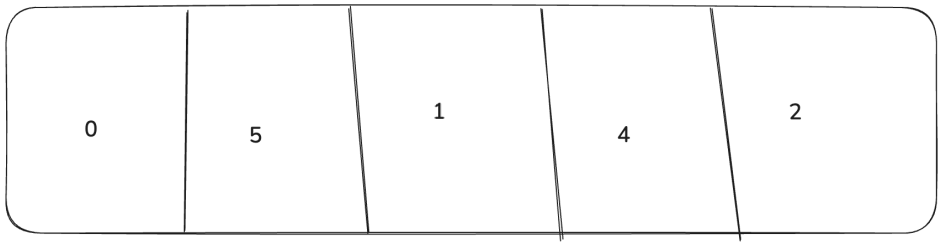
  - In the above, for each query, the rank_i term is the rank of the first relevant item in the list of k predictions
  - Notice that, if we have a more than 1 relevant item, this metric does not care about the ranking of that item
  - MRR is very good when there is 1 relevant item and mAP is in a sense a generalization of MRR, which works for more than 1 relevant item
  - If there is 1 relevant item at most per query, MRR = mAP
  - Note: to understand this, we might want k large to see how far down the ranking list we put the relevant item

Offline Metrics: MRR

- For example, if we have a query and 2 Systems above, the MRR is 1/2 in either case
- However, the System 2 is better, it has more relevant things in the top 5 ...

Offline Metrics: Normalized Discounted Cumulative Gain (nDCG)

- Sometimes, given a list of items, we have not just yes/no marker on the item but a relevance score per item
- In this case, it might make sense to give a reward to a metric not just using the position and relevant / irrelevant but using the actual score we assign
- For example, suppose we rank 5 items as below and we assign a score to each item as below ... Suppose we have a rank score of 0-5 for each item
- For example, the second item is very relevant but the first item is completely irrelevant



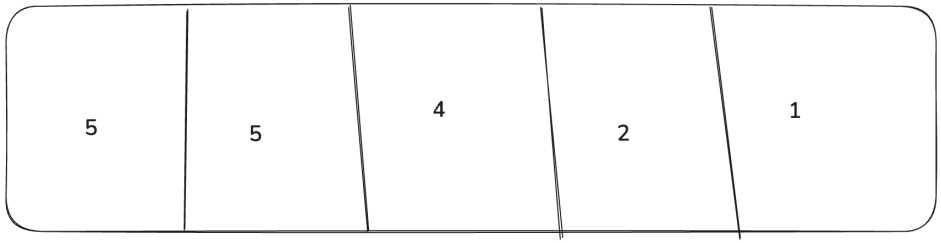
Offline Metrics: nDCG

- Given this, we define the Discounted Cumulative Gain at 5 as

$$DCG@k = \sum_{i=1}^{k} \frac{rel_i}{\log_2(i+1)}$$

- For the above, the value we get is 6.151
- This metric uses our score to reward; there is some theoretical justification for the logarithms
- For any query, the metric value can be all over the place however, so it is better for us to use a normalized version of this metric

Offline Metrics: nDCG



- Suppose the IDEAL ranking scores for this query are as above
- We can compute the DCG of this, and call it IDCG
- For this example, we would get 11.4
- Given this, we define the Normalized Discounted Cumulative Gain as

$$nDCG = \frac{DCG}{IDCG}$$

- For our example, we get $nDCG = 6.151/11.4$

Offline Metrics: AUC

- As a final metric, recall we have AUC given by

$$\text{AUC} = \frac{\sum_{i \in \mathcal{P}} \frac{\sum_{j \in \mathcal{N}} 1_{\{g(q,v_i) > g(q,v_j)\}}}{|\mathcal{N}|}}{|\mathcal{P}|}$$

- For a query, can compute this metric and notice we can use a score which associates a query to each video
- For each query, there might be 5 relevant videos or 100 so $\mathcal{P}$ and $\mathcal{N}$ are naturally defined
- I.e., we don't need to worry so much about k, this naturally will take care of itself
- Of course, an average AUC of 100 % across all queries will tell us that our System is doing very well - all positives are ahead of all negatives
- What k do we display? Maybe we have a dynamic one or an average one given $|I(q)|$ on average

Online Metrics: CTR, CVR, CVS?

- All the metrics we've discussed above are Offline Metrics: we have a machine learning model and we want to get it to rank things as well as possible
- The analysis we do is done in a research / notebook setting, using historical data ... What happens once we deploy?
- In that setting, a user types a query and then gets list of results
- ... How do we tell if the results were good? How do we build a larger system that uses the ranking model we evaluated?
- Let's first look at the basic metrics which we would use to evaluate System-level performance

Online Metrics: CTR, CVR, CVS?

- When we present the user items, they can click on the results
- This gives rise to CTR, click-through-rate, which means

$$CTR = \frac{\text{Number of Clicks}}{\text{Number of Impressions}}$$

- There are many other ways to also engage
- ... For example if we present items to buy and the user buys them, we can the conversion rate

$$CVR = \frac{\text{Number of Conversions}}{\text{Number of Impressions}}$$

Online Metrics: CTR, CVR, CVS?

- But impression does not mean we viewed the item ... We also have
$$CVS = \frac{\text{Number of Conversions}}{\text{Number of Views}}$$
or
$$CVS = \frac{\text{Number of Conversions}}{\text{Number of Clicks}}$$
- For the above, we can use cookies to track if a user viewed the page, clicked, etc
- For videos, we might also have average watch time or average completion rate
- If we present k items per query, all of these metrics might be computed at different k so we can see the effect of results at the top and in the middle of the presented list
- The idea is that these metrics are correlated to offline metrics, but the correlation is not always anywhere close to 1.0
- Moreover, enough though these are easy to measure, we might be interested in more complicated metrics, such as average watch time not just click though rate

Outline

- Recommendation Problems
- Metrics
- Design Youtube Video Search

Problem

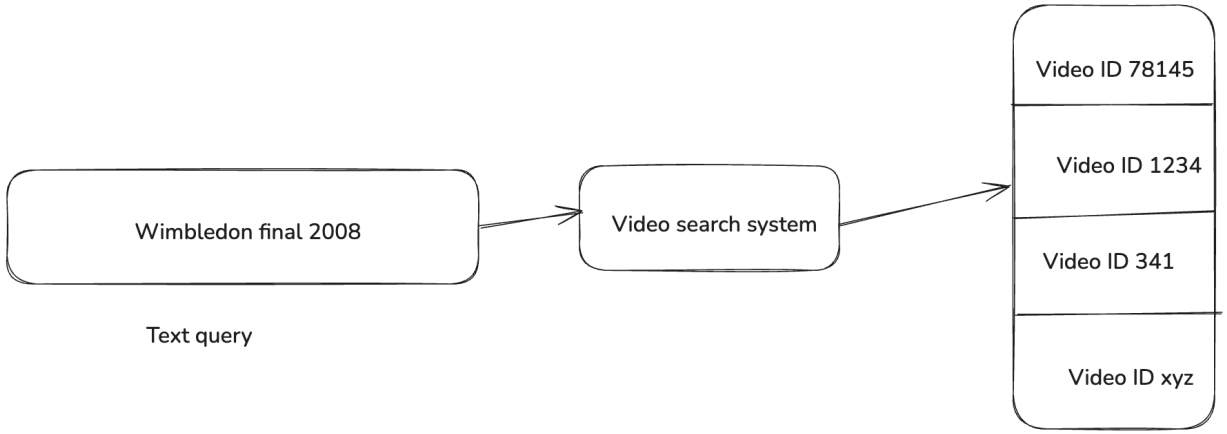
- So back to the concrete problem: Given a user and a textual search, we want to present the best videos to this user
- Some assumptions
  - There are 1 M videos
  - There are 1 M users
  - We have lots of information about interactions
  - I.e. When a video is watched we know its name, if it was clicked, finished or what percentage was watched, we know the age of the video, what channel it is displayed in, etc

Problem: Nail down what is going on

- Usually, in situations like these you want to ask some questions to clarify ... For example
  - Question: Is the input text-only, or can users search with an image or video?
  - Answer: Text queries only
  - Question: Does the platform serve more than videos?
  - Answer: Just videos
  - Question: Do I have information like video title and description?
  - Answer: Yes; you have this and also the the video content
  - Question: Do we have training data?
  - Answer: Yes, let's assume we have about 10 M positive pairs (video id, text query)

Problem: Nail down what is going on

- Usually, in situations like this you want to ask some questions to clarify ... For example
  - Question: Do we need to personalize results for different users given the same query?
  - Answer: No, but do the best you can
- Summary goal: Design a search system for videos ... The input is a text query, the output is a list of videos that are relevant to the text query
- What is our objective?
  - Our objective is to get the most relevant videos for a user given a textual search



Strategy and speed considerations

- We have a problem ...
  - For a given search and video (+ features), we need to say which videos would be the top choices (top 10, ranked) and we need to do this fast
- "Fast" means we need at a high level a quick way to map a query and some features to a top 10 list of videos
- Finally, notice that this is not personalized - we have no user information to worry about

Figuring out how to represent the query and the videos

- So given a query, we want to match videos which are similar to this query
- How can we represent a query
- How can we represent a video?

Representing the query

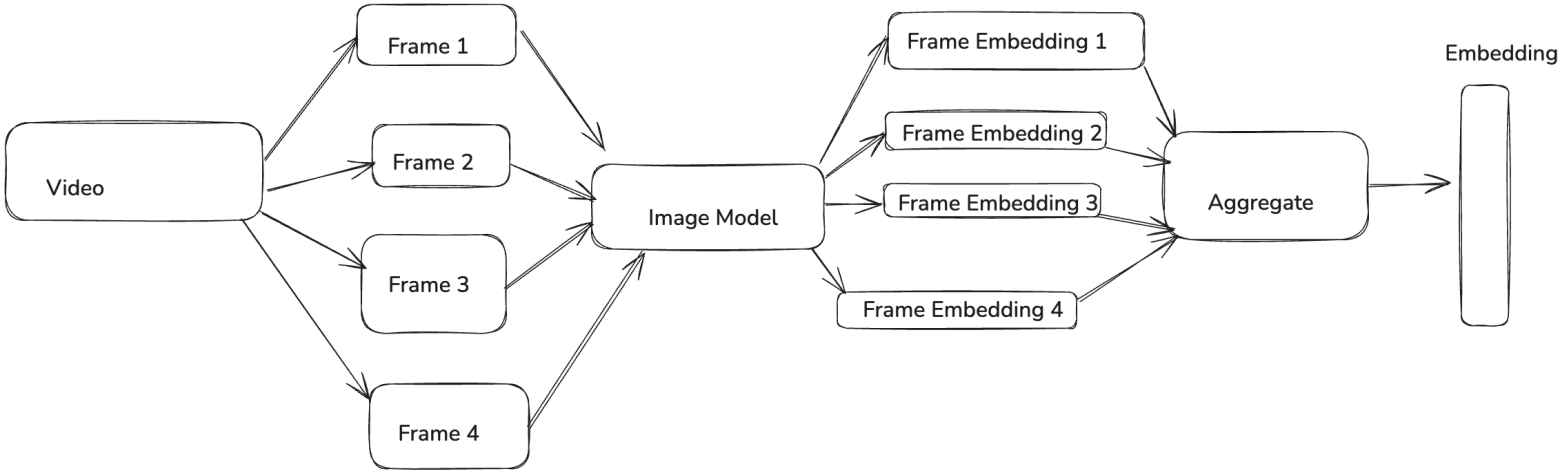
- The query is text, and we know several ways to represent it
  - We can use Word2Vec - We take the embeddings for each token / word in the query and average over them
  - We can use SentenceBERT - We take the CLS token and use it to represent the sentence or we average over all the other tokens in the sentence
- Which one is better?
  - Word2Vec - Very quick, but no semantic meaning ... Words like "bank" will always have the same embeddings and if a sentence is "a b c d" or "d c a b" we get the same representation via averaging
  - BERT - Better, contextual ... But slower ... However, we can use variants of DistilBERT which are small and still give strong representations

Representing the videos

- Videos have a text part and a video part
- For the text part, we can use any of the methods we used for the query
- For the video part, there are a few options, but we want to summarize videos via an embedding
  - One idea: consume the entire video, and create an embedding to summarize it
  - This idea uses 3D convolutions and Transformer models
  - This is expensive, as we need to consume the entire video

Representing the videos

- Another idea to represent the videos:
  - Another idea: grab frames and use these frames to represent the video
  - At a high level, this grabs frames, processes each image with a CNN or some other Transformer, and then averages the embeddings of the frames to get the overall embedding for the video



Making sure the representations are close

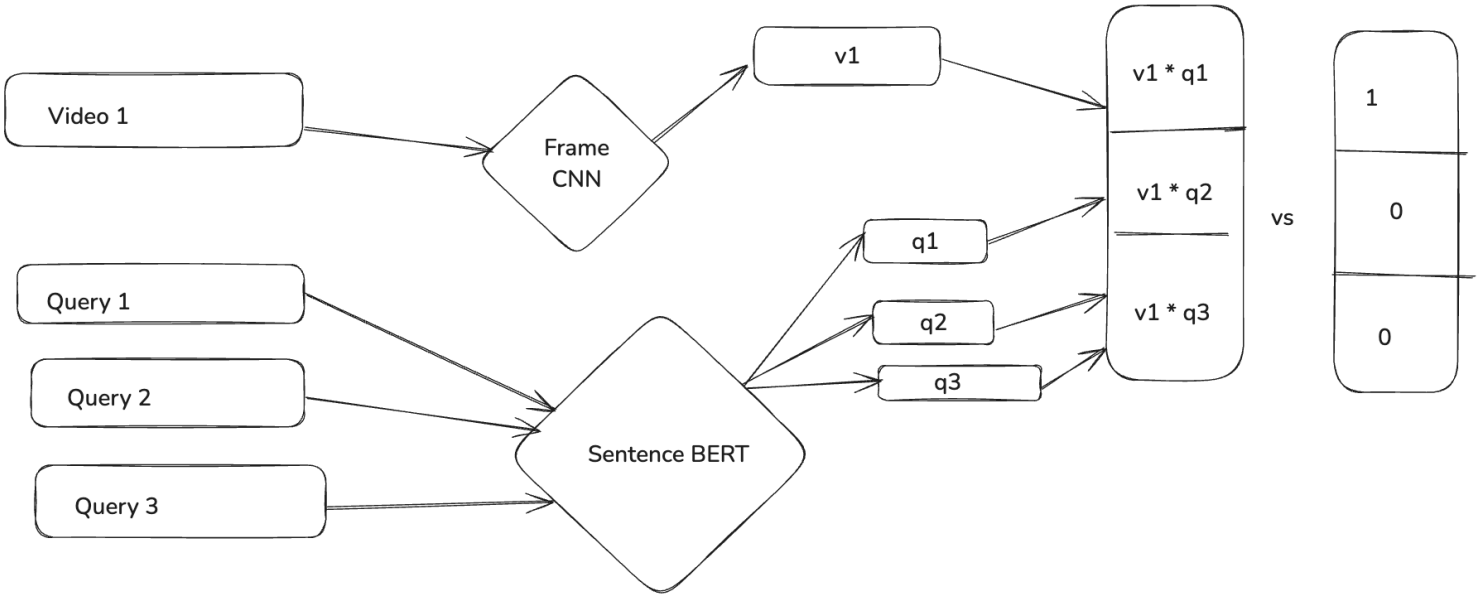
- For the query and the text representations of the video, what do we know?
- Well, we know that generally similar query - text descriptions have high cosine similarity and low ones have a low similarity
- This was the idea of SentenceBERT and we saw that shallow models like Word2Vec also are OK to represent sentences
- What about the video embeddings vs the query search embedding?

Making sure the representations are close

- Given query text, we have a model that produces an embedding
- Given some video frames, we have a model that produces embeddings
- The above two models are different, and we need to modify these models to yield embedding such that
  - If the video and the query are the same, the inner product is large
  - If the video and the query are different, the inner product is low
- What do we have and how can we use this?

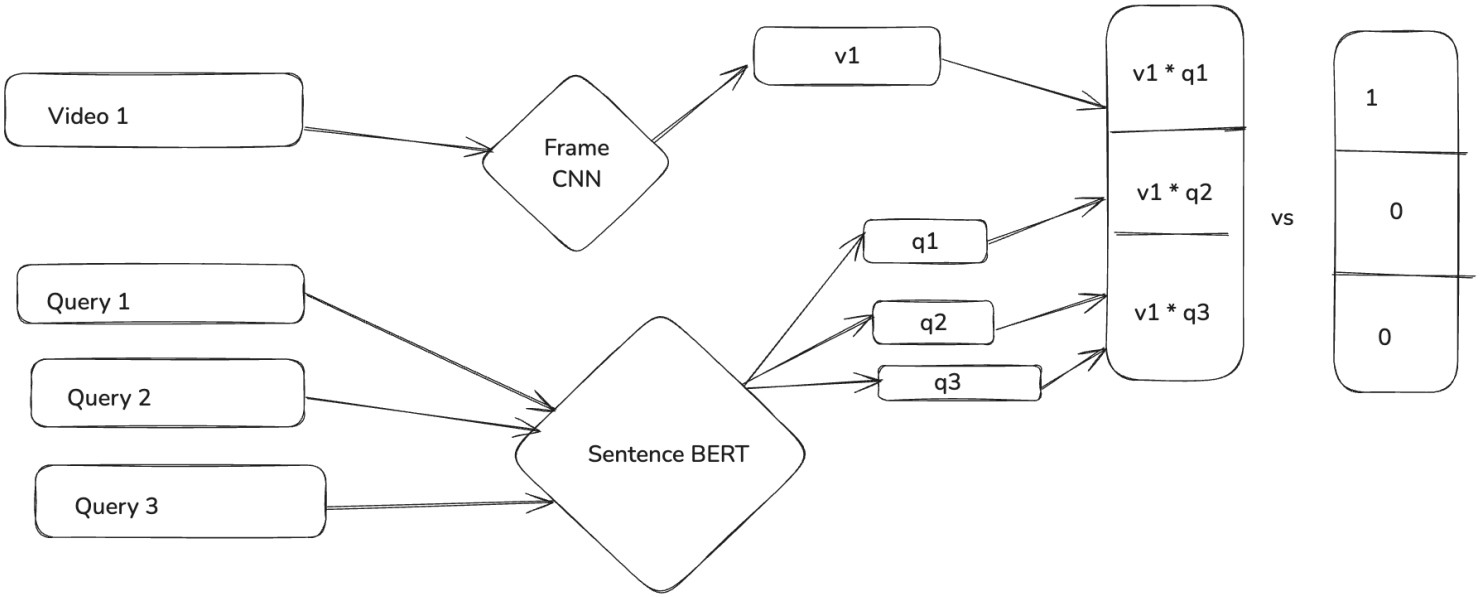
Making sure the representations are close

- We have a set of ordered pairs $\{(q_i, v_i)\}_{i=1}^{N}$ where each query is associated with a video (we collect such data or synthesize such data)
- We can assume that most of the time q_i and v_j are not associated if $i \neq j$
- Suppose we take 1 video and 3 queries and 2 queries are unrelated to the query while q_1 and v_1 are related
- In this case, we might have something like below



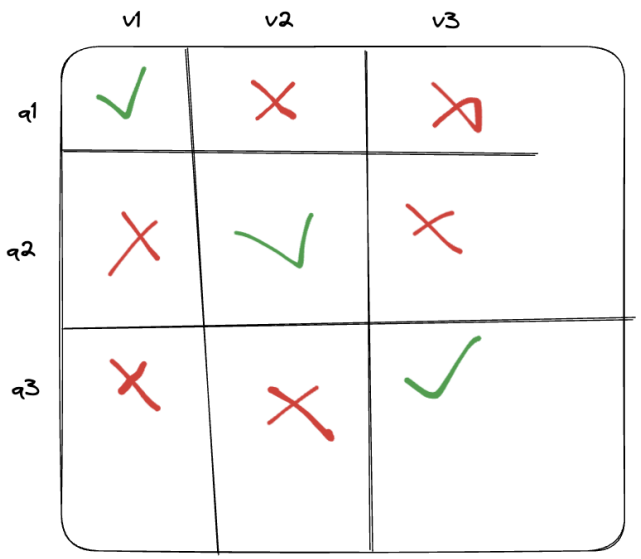
Making sure the representations are close

- From the below, we have that $q_1^T v_1$ needs to be high
- $q_1^T v_2$ and $q_1^T v_3$ need to be low
- We can consider the graph below and back propagate into the CNN and BERT models by doing a Cross-Entropy of the above logits vs (1, 0, 0)
- Fundamentally, this will change the 2 models to product text embeddings which are close to video embeddings when appropriate and far when they are different



Making sure the representations are close

- This last setup is similar to what we saw for the BiEncoder
- We start with positive pairs, assume anything off diagonal is negative, and then come up with examples we can learn from
- For the example we consider this is the first column of the picture below



How do we use these embeddings

- So now we have
  - Video embeddings which are close to query embeddings when appropriate ... We've placed video and text embeddings in the same space
  - Text embeddings which are naturally close / far now from similar and dissimilar embeddings

2 SentenceBERT Models?

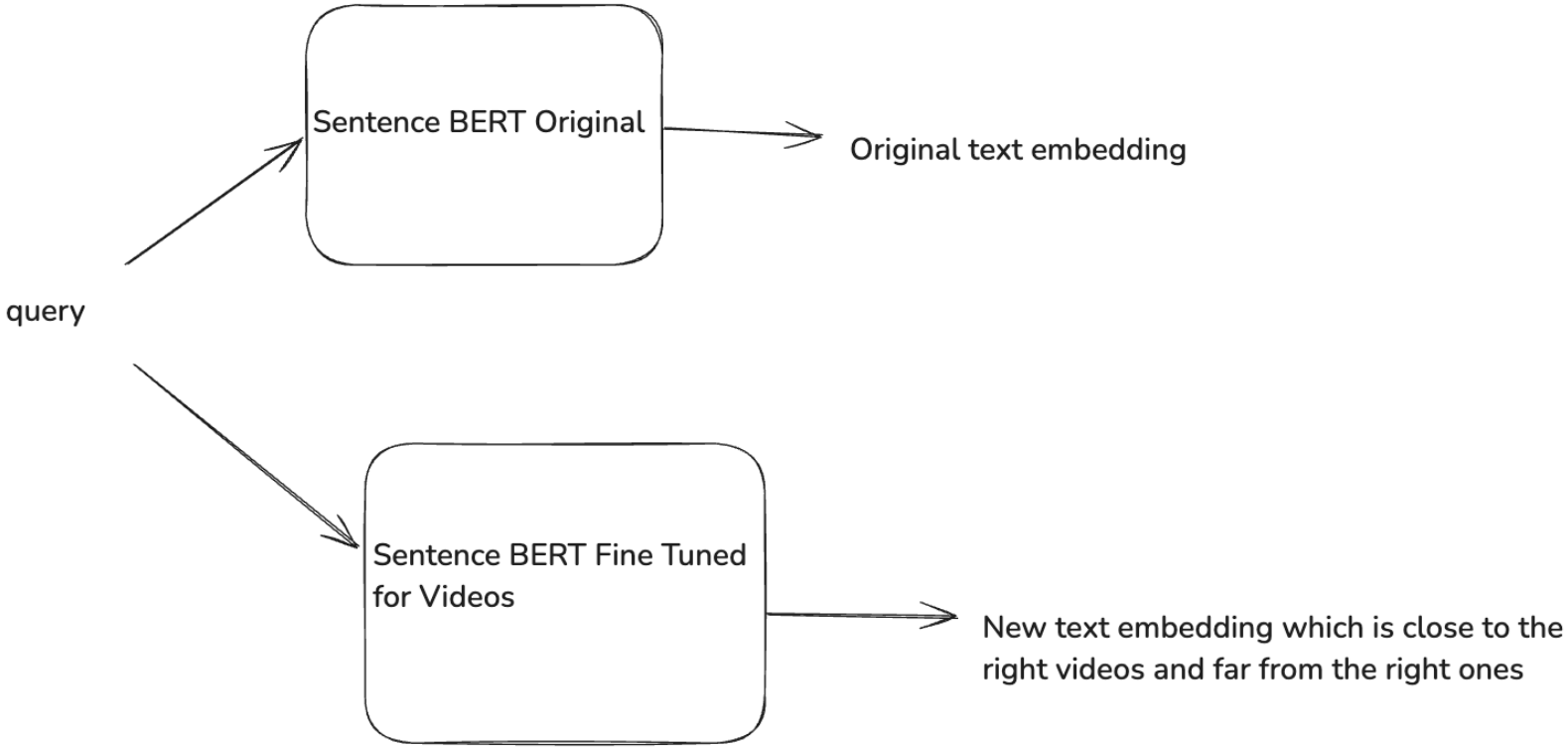
- What else? Well, we need to be careful ...
- Notice that we've changed SentenceBERT such that now a query embedding is close/far from a video embedding but
- We need to make sure that the "old" properties of the SentenceBERT model still hold: similar queries have similar embeddings and dissimilar ones have low inner product
- To make things even easier, we can have a normalization layer above so that we don't work with inner product but cosine similarity ... I.e. the embeddings models produced are normalized

2 SentenceBERT Models?

- For the SentenceBERT issue we can
  - Have a complex objective which still preserves the textual properties we started with
    - For example, have the objective or data have query-video pairs and query-query pairs
  - Make another copy of SentenceBERT and fine tune just this one as above ... This way, in some sense, a query has a "text" embedding and a "text-video" embedding
  - For this last idea, the query travels through both models

How do we use these embeddings

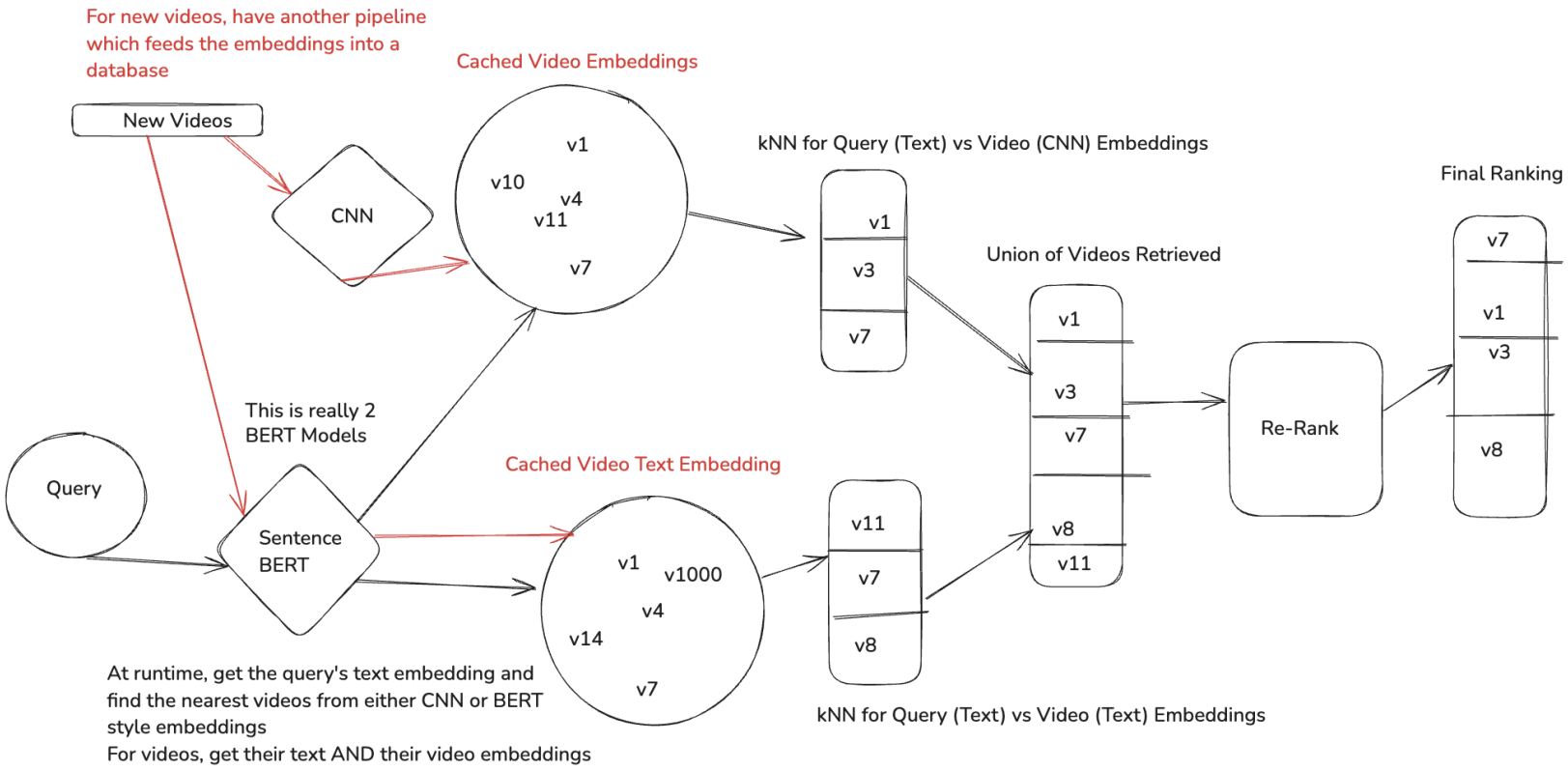
Each query text can get "two" embeddings ...



How do we use the embeddings?

- Crucially, note that for the videos we can have a service which computes text and video embedding and we can store these vectors them in a database
- At run time, we just compute the query's embedding(s)
- For a query, we get videos which are associated with the query-video text and and query-video content similarities
- For example, we can score videos using a combination of $q_{text}^{\mathsf{T}} v_{text}$ and $q_{video}^{\mathsf{T}} v_{video}$
  - Use $q_{text}^{\mathsf{T}} v_{text}$ to get a list and $q_{text}^{\mathsf{T}} v_{video}$ to get a list and them take the union
  - Use $\frac{q_{text}^{\mathsf{T}} v_{text} + q_{video}^{\mathsf{T}} v_{video}}{2}$ and just get 1 list
- Once we have a set of videos, we can then re-rank them using a heavier model
- In the above, q_{video} is what is coming out of the fine tuned SentenceBERT

So what might a full flow work like?



Re-Ranking? What else?

- In the last picture you see a ranking step
- The idea of this type of design is that we have a huge number of videos to select, and we first want to focus on retrieval and high Recall so that we get a set of relevant videos for our query
- Now, once we have a manageable set of size 100 (say), we can use a heavier model to *refine* our results with ranking ...
- At the ranking stage, we focus on Precision as the key metric

Anything before we rank? What else?

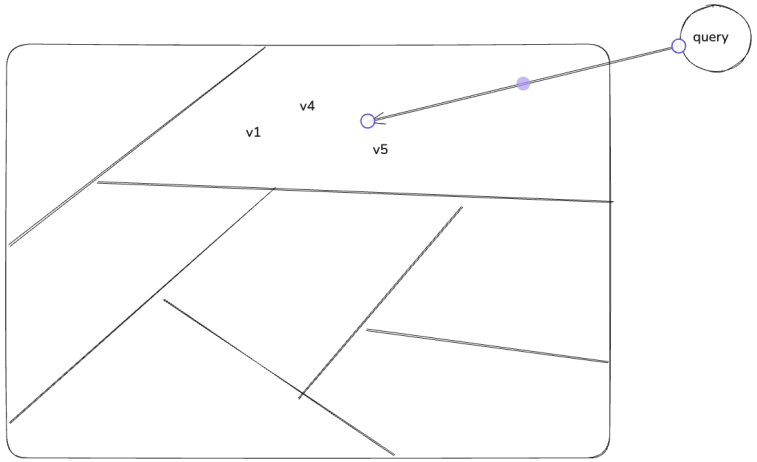
- The architecture we used was nice for a few reasons
  - Essentially, we used a BiEncoder for relevance - this is a Two-Tower architecture which produces embeddings such that a search embedding is close to the video embedding
  - This type of architecture is good because we can cache / save embedding and at runtime we just need the query embedding
  - A set of videos might be huge and a standard search $O(N)$ to find the 10 best, but there are methods to do this much quicker in $O(\log N)$ time

Candidate Generator and kNN

- At this step, we need to find, for a search, the closest videos
- Here, closest means the videos with the highest inner product to the user's embedding
- Thus, we score videos via a score like $q^\intercal v$
- Effectively, we want to find the k nearest neighboring videos given a user in terms of their dot product
- The problem is that finding the nearest neighbors this way, referred to as maximum inner product search (MIPS) across one million videos can take prohibitively long when serving candidates in real time to user
- Specifically, if there are $|V|$ videos and each vector has dimensions d, finding the top k will be $O(|V| \log kd)$ (use a max heap)
- So, we'll need a way to trade off the accuracy of the nearest neighbors search in favor of a reduced latency ... We need to get rid of the $|V|$

LSH kNN idea

- This type of issue falls under the umbrella of approximate nearest neighbor search
- The main idea of most approximate kNN algorithms revolves around breaking up the embedding space into regions, for example, locality sensitive hashing (LSH)
- The main idea of LSH is to partition the embeddings by their similarities and then when searching for nearest neighbors, look only at the partition that the input example falls into
- At runtime, this prevents the need to search through all of the video embeddings and instead isolates the search to a partition which will contain similar embeddings / videos



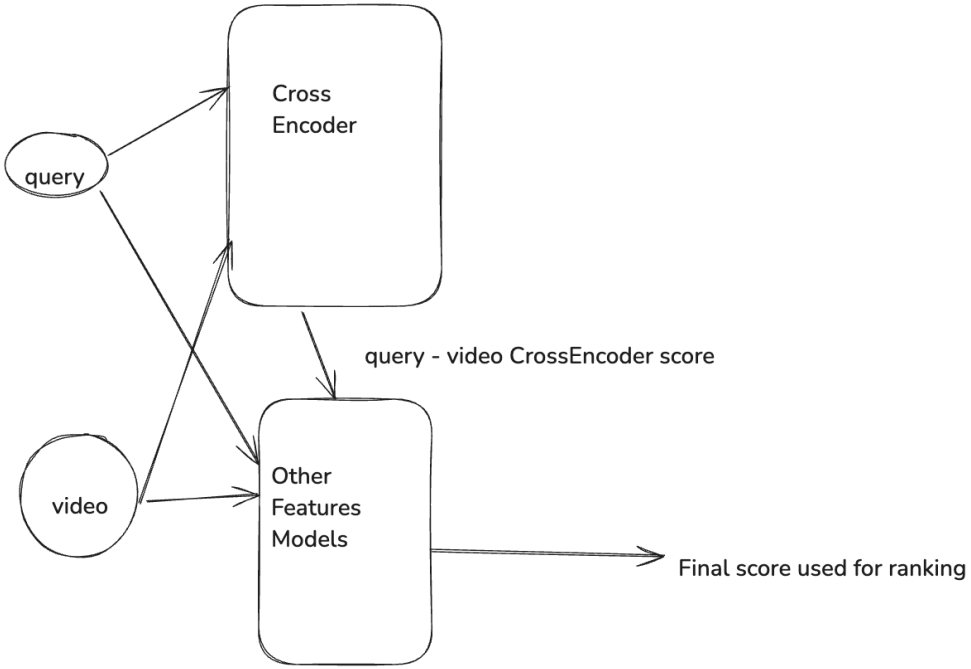
How does kNN work?

- We have $|V|$ videos and we can't find the top videos for a query embedding blindly because this will be $O(|V|)$ which is too expensive
- Approximate kNN algorithms try to reduce the above complexity to $O(\log |V|)$ thereby allowing much faster retrieval
- High level ideas?
  - Locality sensitive hashing would map each video to a hash ... At runtime, the query is projected to a bucket and we only look for videos inside of that bucket
  - Tree based methods also work, and they partition the space of videos into regions ... At runtime, the query is projected to a region and similarly we just consider videos in that region
  - Finally, we can also group videos by similarity via clustering ... At runtime, we look at the cluster the query belongs to and then just search that cluster
- For all the ideas above, the methods are tested and available to you, so you can use things like FAISS or SCANN quite easily ... The error rates on these algorithms is quite low

Re-Ranking? What else?

- Some ideas on the final ranker
  - At this step, we can also use a CrossEncoder so that for each candidate video we get a score when we feed in the *pair* (q, v) where q is the query v is the video ... We do this for all videos we need to consider
    - For example, if we have 100 videos we might break up the 100 into batches of size 10 feed these batches to servers which host the CrossEncoders and then get 100 scores and rank the videos based on these scores
    - Here, we batch since we want to distribute the load but this might not be necessary with the right model + GPU
    - I.e. if the model large, we might need this ... If it is small, we might not

Re-Ranking with a CrossEncoder and more



Re-Ranking? What else?

- Some ideas on the final ranker
  - For each video, we also know features such as: historical CTR EMA for 3, 5, 7 days, EMA of likes for 3, 5, 7 days, differences between EMAs, age, topic, etc
    - We could also have a MLP which takes in the query embedding, the video embedding, a set of normalized video features, and finally produces a score (maybe a probability of click)?
    - This is another idea but notice that we have a lot of features here but many fewer videos
    - There are other cases: we can have multiple rakers where the rankers get more and more "heavy" (i.e. first we have a Logistic Regression, then an MLP / CrossEncoder hybrid but this last ranker runs on very few videos)
  - After ranking, we can even have another phase that boosts new videos up, ensures diversity, etc - these could be heuristic rules
  - This last step is crucial also!

What if we wanted personalization?

- In this case, for two users who search the same query, we expect that the answers might not be the same ... I.e. We want to return answers that depend on the user ID
- In this case, then, we need to build for each user a feature vector so that we can incorporate information into the model's ranking
- One idea: We have a system such that for a search query we return a set of relevant videos ...
- We could ...
  - Use our previous system to retrieve a set of relevant videos regardless of the user
  - At the ranking step, we can impute user features such as age, location, sex, historical CTR, historical logins etc and use these features to rank videos
- But, ideally, we don't just do this at ranking time ... Ideally we have a set of user embeddings per user and we use this at retrieval time

What if we wanted personalization? More topics for next time ...

- We'll need a way to figure out user embeddings
- What about new users? This is the cold-start problem
  - For our previous analysis, we could have new videos but we just need to push these through the CNN and save the embeddings
  - Moreover, if we are not sure, we can use some sort of heuristic .. For example:
    - Find the topic of the video
    - Use the average video embedding across videos in this topic
- What about bias?
- What if the model is large?
- What about click-bait videos?
- What is Sparse search? I.e. How else can we search beyond just using embeddings and why should we even bother?

References

- Matryoshka Embeddings
- Deep Neural Networks for Video Ranking
- AP, Recall, Precision, MAP, Etc.